



EG1311 DESIGN AND MAKE

B13 TEAM 1 REPORT

NAME	MATRICULATION NUMBER
Seenivasan Siva Sanjanaa	A0320131E
Nguyen Anh Duy	A0325620N
Janice Yeo Sze Min	A0321402B
Kevin Wong Hong Ming	A0322635J
Lim Ying Tong	A0331142X
Skandesh Maniprakash	A0334864W

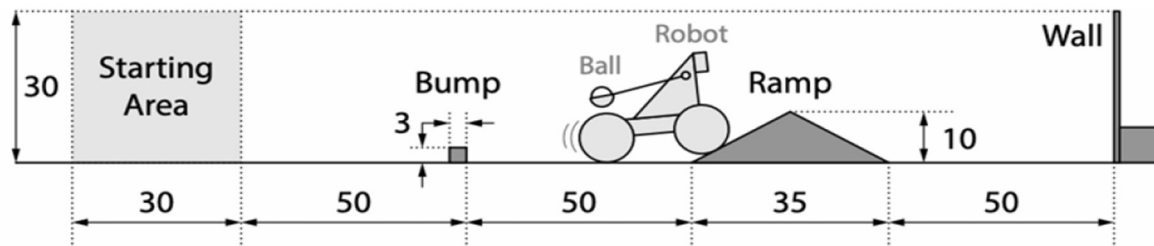
Table of contents

1. Core idea of project	3
2. Ideation	3
3. Prototyping process	4
4. Final design	7
5. Key takeaways	7

Appendix

Final CAD Rendering	8
Picture of Final Robot	8
2D CAD Rendering of Components	11
TinkerCAD Diagram	13
Arduino Source Code	15

Core idea of project



The objective of this project was to build a self-powered robot capable of navigating an obstacle course comprising a 3cm bump, a 35cm ramp, and a 30cm wall, stopping within 5cm of the wall and launching a ping pong ball over it. Building upon the Week 1 template robot, our team took an iterative approach, identifying and resolving hardware and software challenges at each stage of the course. This report documents the solutions we developed and the lessons learnt throughout the process.

Ideation Stage

The ideation stage began with analysing the project requirements and the limitations of the template robot provided. Rather than designing a completely new robot from scratch, our team chose to build upon the template robot by identifying its weaknesses and exploring possible improvements.

During initial discussions, we considered both 2-motor and 3-motor configuration. While a 2-motor design could reduce weight and improve efficiency, it was expected to provide less traction and control. In contrast, a 3-motor configuration offered greater driving force and improved stability, particularly when overcoming the ramp and bump. These factors guided the selection of the final motor configuration in the later stages of prototyping.

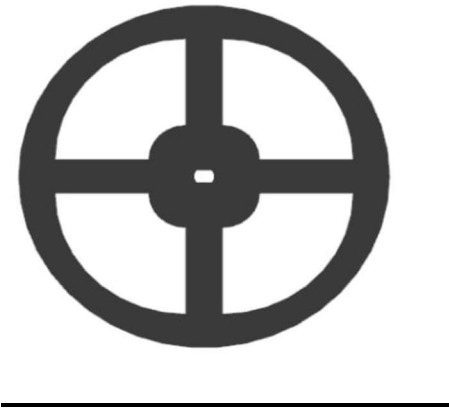
In addition, we identified traction as a critical factor for the robot to overcome the bump and ramp. Therefore, we considered adding rubber mat-lined to the wheels to increase friction. Weight distribution was also considered, as the robot was required to carry both a 9V battery and six dry cells, which could significantly affect its balance and performance.

We also anticipated challenges such as wire entanglement and structural instability when using lightweight materials. As a result, our design focused on balancing weight reduction with sufficient structural strength while minimising interference during operation.

Overall, the ideation stage established key design considerations that guided the subsequent prototyping process.

Prototyping process

After finishing the template project, we decided to increase the diameter of the wheel for the first iteration, as we want the robot to easily clear the bump and ramp. We also decided that our wheels should be made out of acrylic, as it is the most rigid material at that point.

Wheel Prototype	
100mm diameter, 10mm spokes	100mm diameter, 9.5mm spokes
	
-With the help of rubber mat lining wrapped around the wheels, the robot was able to clear the obstacles -However, the weight of this design is a problem. There were still unnecessary parts of the design that we can remove to optimize it.	-Addressing the 1st design's problem, we came up with this new design. -3 spokes proved to be strong enough to withstand the load -Removed 1 spoke, redesigned the center, thinner outer rim.

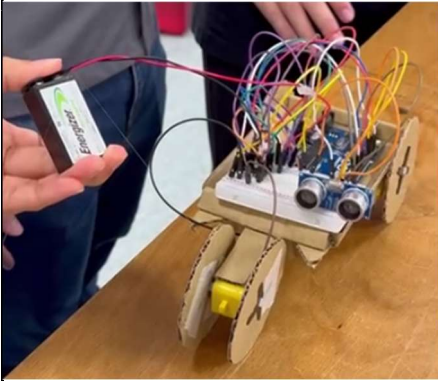
Catapult design

For the catapult design, we prioritised sturdiness and weight efficiency. Initial testing with a single ice cream stick arm proved insufficient, as the torque generated was too low to clear the wall. Applying the principle that torque is the product of force and distance ($\tau = F \times d$), we extended the arm by attaching two ice cream sticks back to back, increasing the moment arm and generating sufficient torque to launch the ball over the wall. For the ball holder, a simple masking tape pouch was used, providing the optimal balance between weight and structural integrity



Robot Prototyping

Prototype 1	
	Observations



4 wheels(cardboard) , 3 motors, cardboard chassis

How we tested

The wheels slipped on the ramp, confirming that a higher friction material such as rubber matting was needed.

The chassis bent noticeably at the centre under the full battery load, confirming that cardboard was insufficient as a structural material.

The robot struggled to cross the bump due to uneven component placement shifting the centre of mass

-Cardboard wheels lacked friction, preventing the robot from crossing the ramp.

-Cardboard chassis was too flimsy to support the weight of the batteries and servo motor.

-Uneven weight distribution caused the chassis to bend at the centre, making it difficult to cross the 3cm bump.

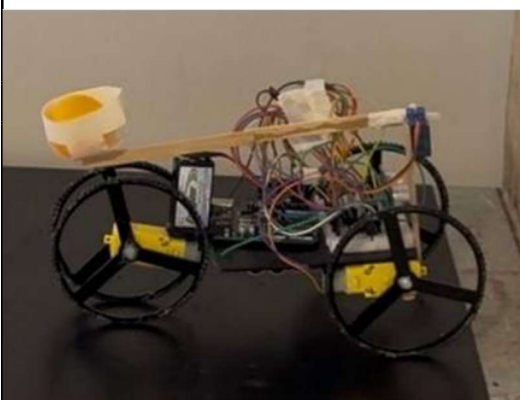
Conclusions

-Needed to add friction to the wheels to improve grip on the ramp.

-Needed to use a different material for the chassis.

Needed to redistribute weight evenly to prevent the chassis from bending.

Prototype 2



4 wheels(cardboard) , 3 motors, cardboard chassis

Observations

- The robot stopped randomly due to the sensor's position.

- Chassis still bent under the weight of the 6 AA batteries, indicating further stability issues.

- Front motors were out of sync, spinning at different speeds and causing the robot to drift off course.

Conclusions

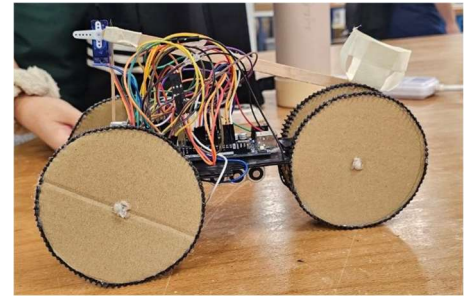
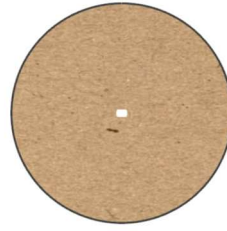
<i>How we tested</i> The robot stopped abruptly at random points, which we attributed to the ultrasonic sensor being positioned too low and picking up false readings. The chassis continued to bend under the full battery weight, confirming that further structural reinforcement was still needed. The robot drifted consistently to one side along a straight path, indicating the front motors were out of sync, leading us to conclude that ice cream sticks could be used to brace the frame and keep the motors aligned.	- Code and ultrasonic sensor placement needed to be revised. - Chassis stability needed further improvement. - Ice cream sticks to be incorporated to reinforce the structure.
--	--

Arduino Code Logic

<pre> void loop() { if (courseCompleted) { stopMotors(); return; } long distance = getDistance(); Serial.print("Distance: "); Serial.println(distance); if (distance > 0 && distance <= WALL_DISTANCE_CM) { stopMotors(); delay(1000); ballLauncher.write(100); // Throw! delay(1000); ballLauncher.write(0); // Reset delay(500); moveBackward(); delay(REVERSE_TIME_MS); stopMotors(); courseCompleted = true; } else { moveForward(); } } </pre>	- courseCompleted to check when to stop the motors - Logic handling: if distance is less than or equal to set wall distance, the stopMotors() function will stop the motors, then wait 1 second before the servo does its work. - The ball launcher will then be written from 0 degrees to 100 degrees to throw the ball - After throwing, moveBackward() function would reverse the robot directions to come back to the start box - After a set duration, the motors would stop, and mark its trip as completed - If the distance is greater than the wall distance, meaning that the robot has not reached the end, the moveForward() function will tell the robot to move straight forward.
---	--

Final Design

The final design consisted of the revised prototype, incorporating cardboard wheels with foam core lining for added friction, as well as ice cream sticks to reinforce chassis stability and motor sync. To reduce excess weight from the previous wheel design, we switched to lighter cardboard wheels, which still proved capable of clearing all obstacles.



During test runs, we found that the robot was unable to reverse, so we revised the code to implement a reversing function. We also repositioned the ultrasonic sensor to the centre of the robot, level with the frame, which improved its reliability and allowed the robot to stop consistently at 5cm from the wall.

To address the remaining chassis and motor sync issues, ice cream sticks were added to reinforce the frame and improve structural rigidity. Additionally, we broke the power rails on the breadboard to further reduce unnecessary weight, making our robot the lightest among all groups. With these changes, the robot was able to successfully navigate all obstacle

Key takeaways

1. Effective Time Management and Decision-Making

Early on, our team's progress gave us a false sense of security, leading to excessive redesigning. We spent significant time debating between a 2-motor and 3-motor configuration, even consulting our TA on the merits of a lighter 2-motor build. However, realizing that constant pivots were time-consuming, we pragmatically stuck with our functional 3-motor prototype. This taught us to prioritize timely decision-making and efficient time management when working under strict project deadlines.

2. Leveraging Team Diversity and Collaboration

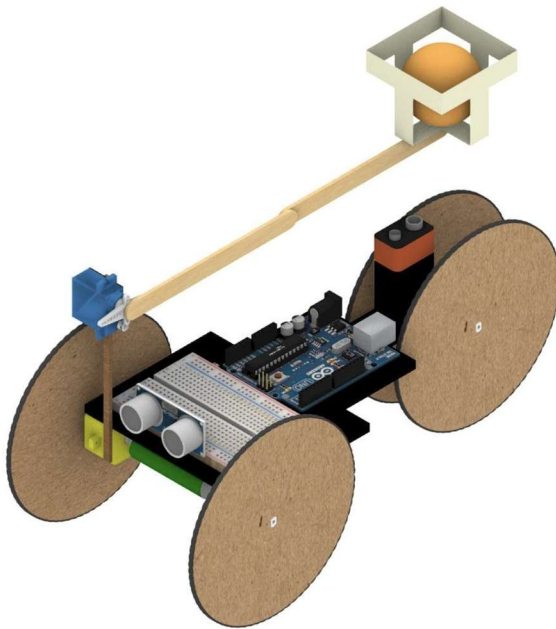
Our team leveraged diverse academic backgrounds to distribute tasks effectively. For instance, our Electrical Engineering major handled circuit design and breadboarding, while the Computer Science major managed coding and logic. This synergy allowed us to solve problems collaboratively and was essential to the successful development of our final prototype.

3. Iterative Testing and Learning from Failure

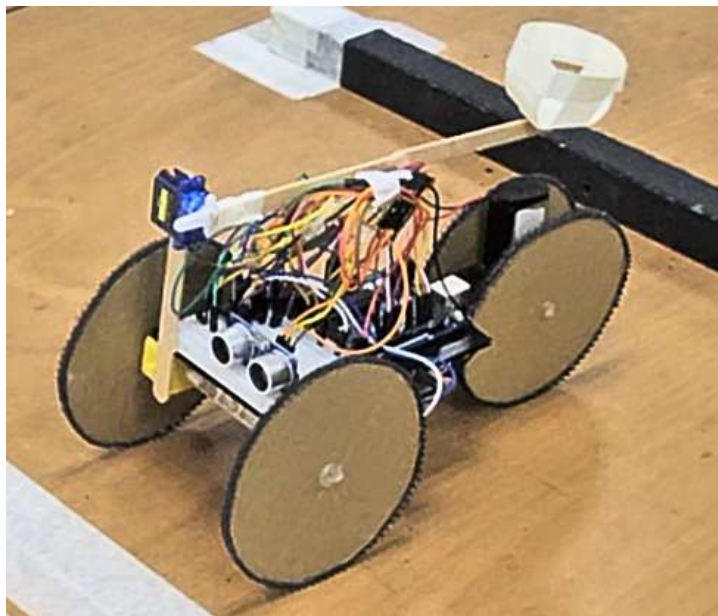
Testing revealed that our initial designs required several iterative improvements. To fix poor traction, we added anti-slip mats to the tires; to prevent mechanical interference, we re-routed and taped down wires that were entangling the wheels. These cycles of troubleshooting and refinement taught us that engineering success isn't about a perfect first draft, but about learning through trial and error to enhance reliability.

Appendix

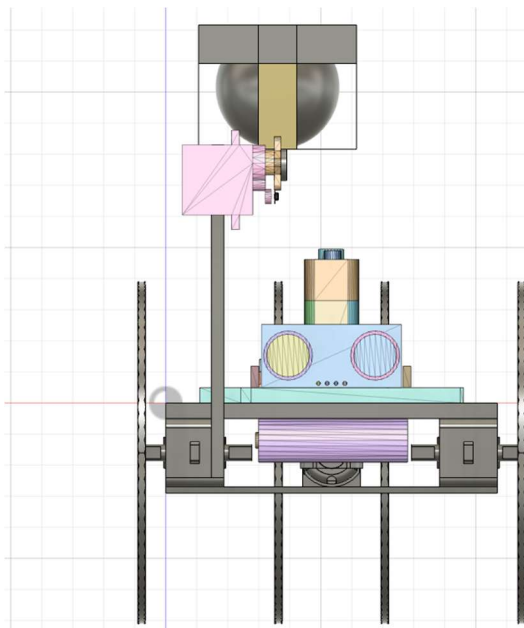
Final CAD Rendering



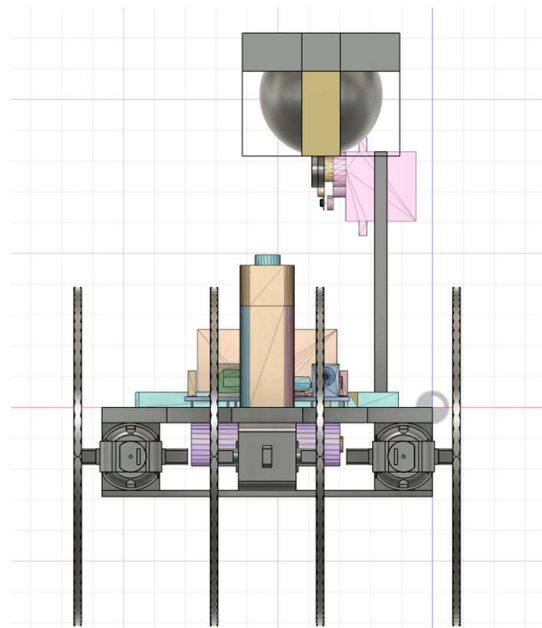
3D Overall View



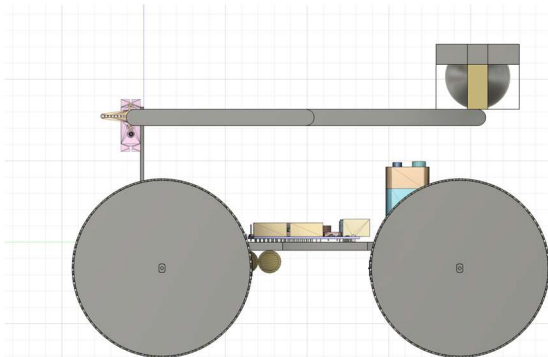
Actual View



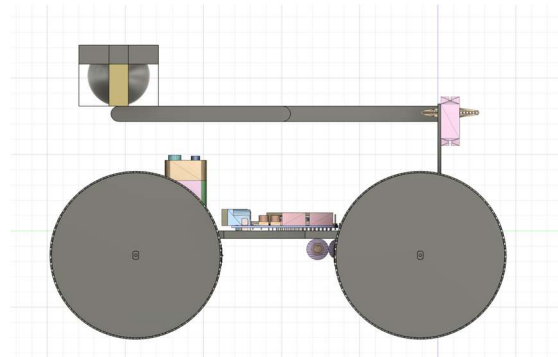
3D Front View



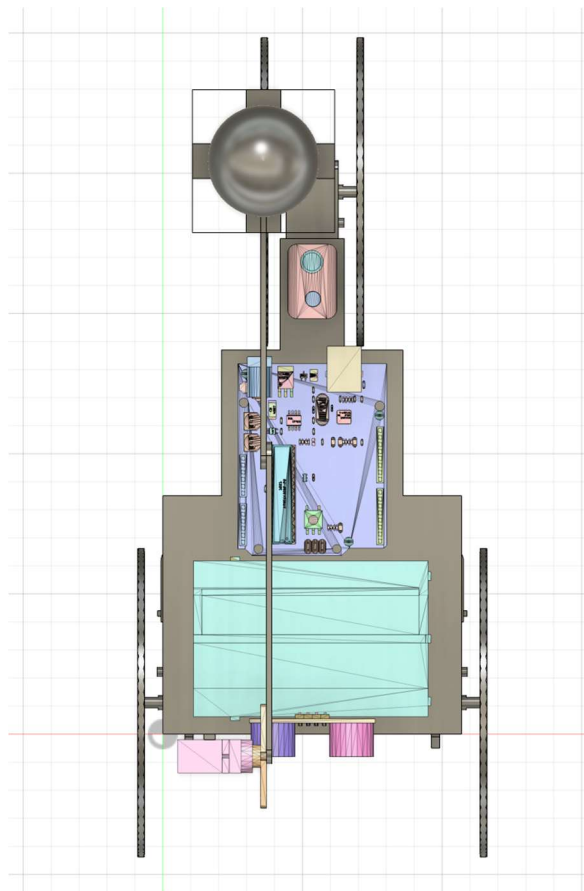
3D Back View



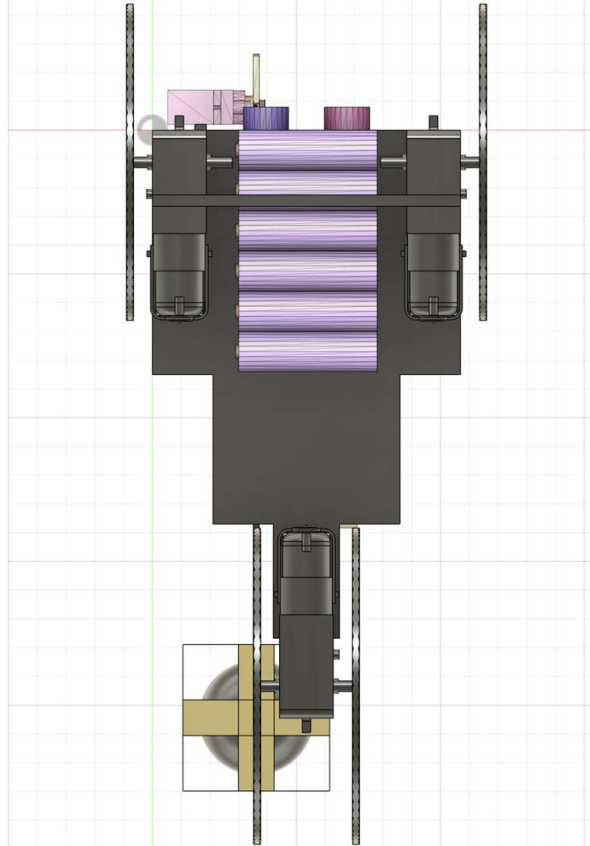
3D Right View



3D Left View

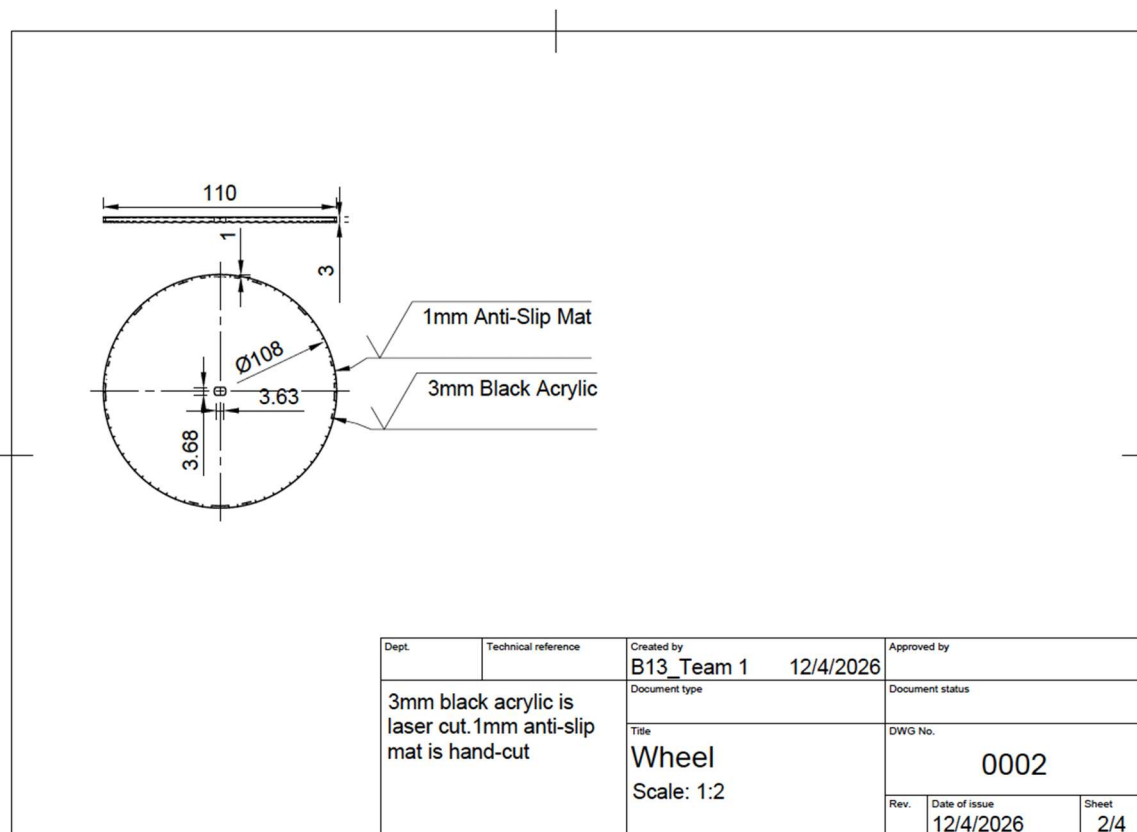
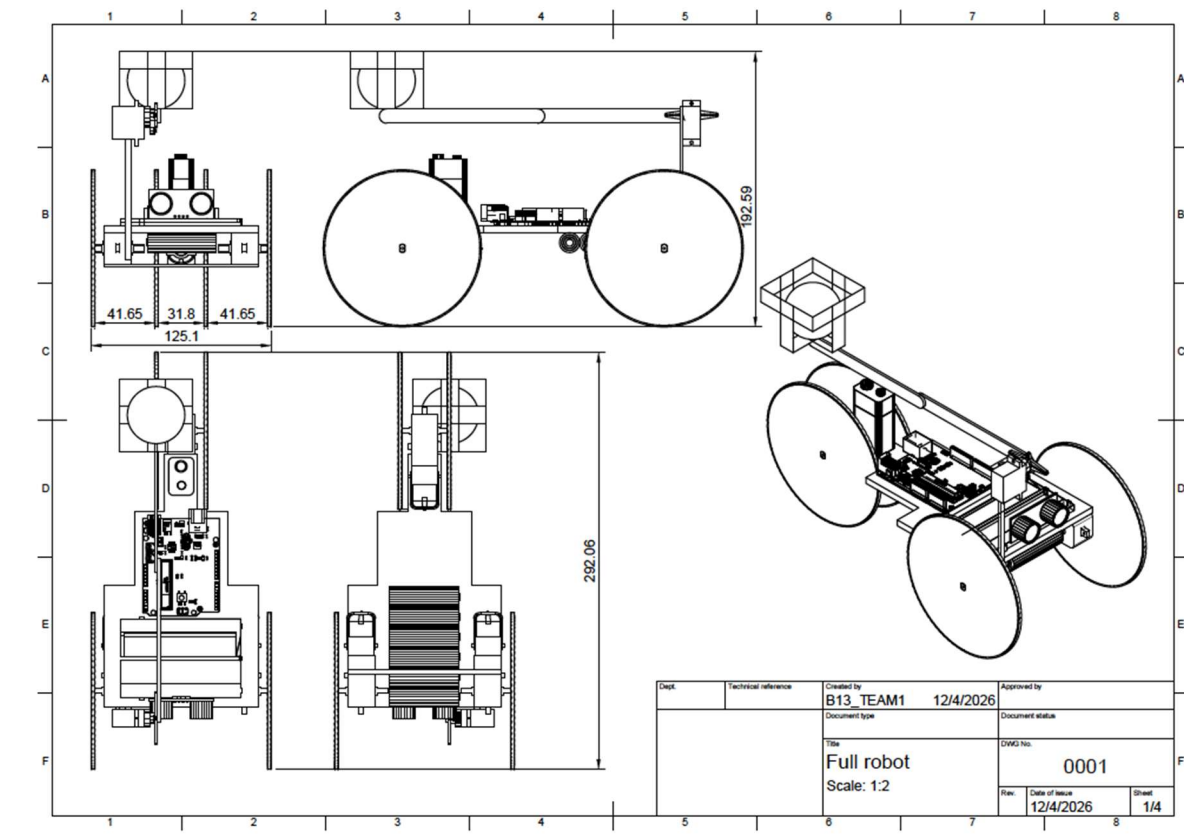


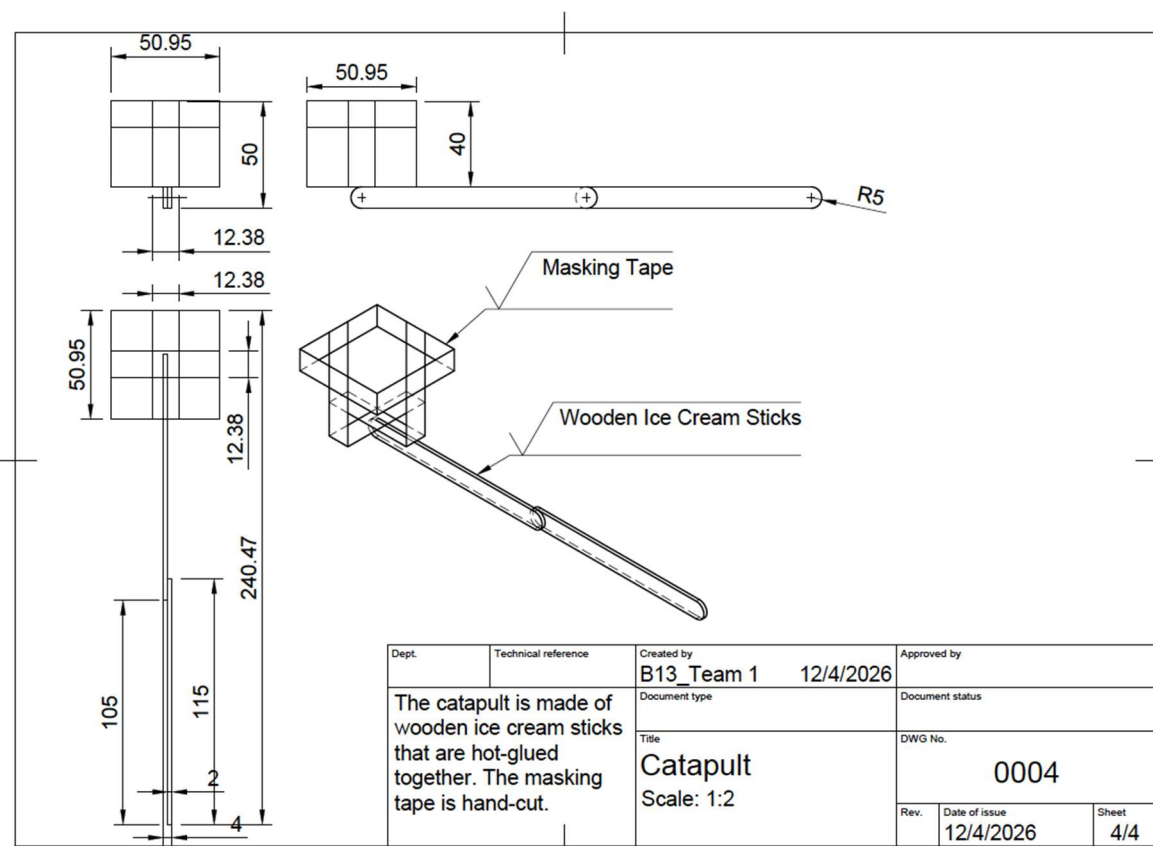
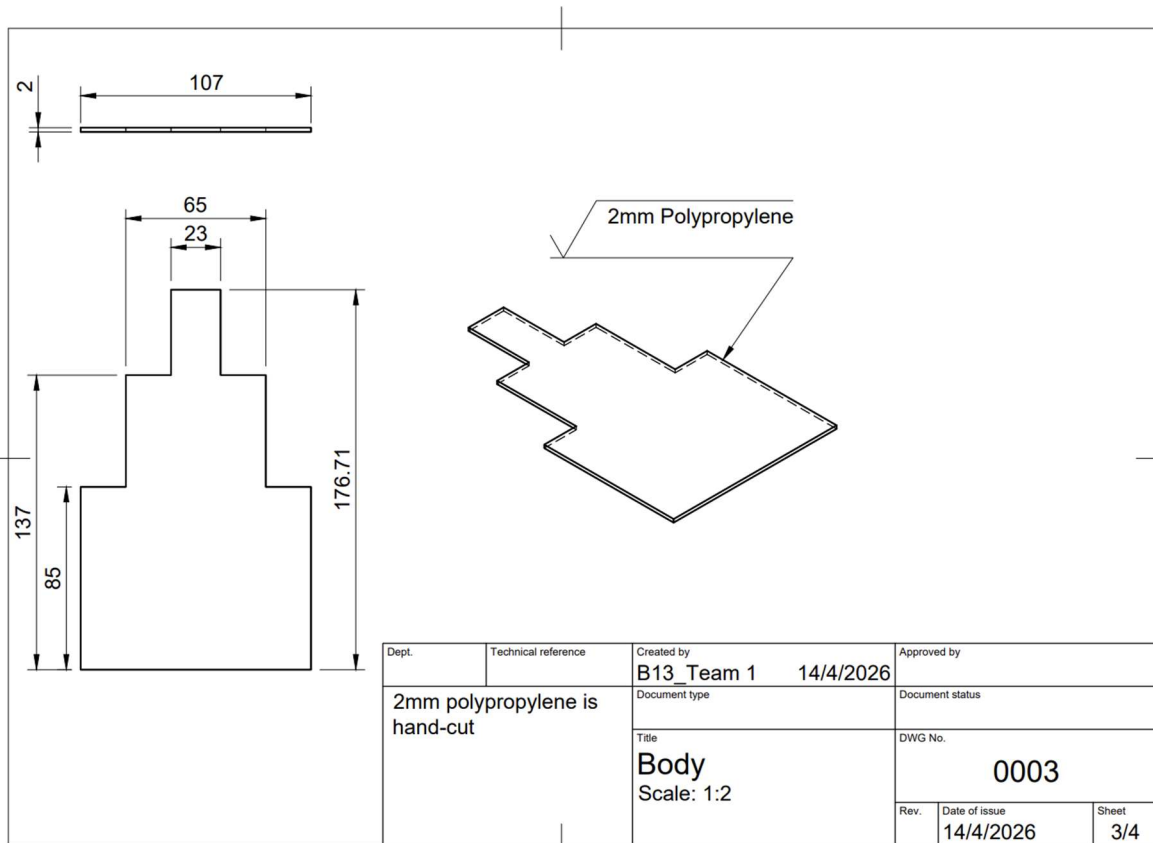
3D Top View



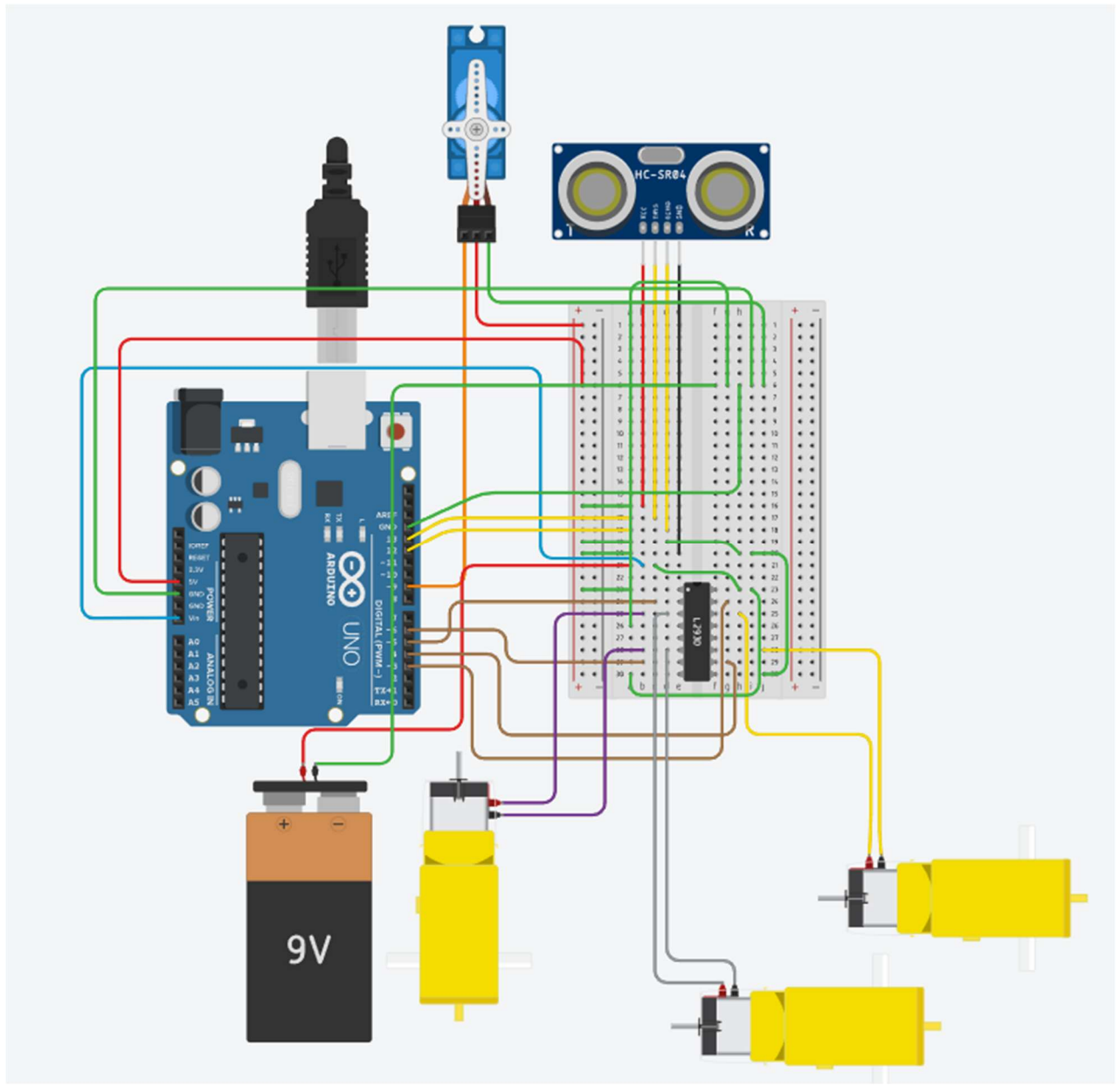
3D Bottom View

2D Rendering of the Components

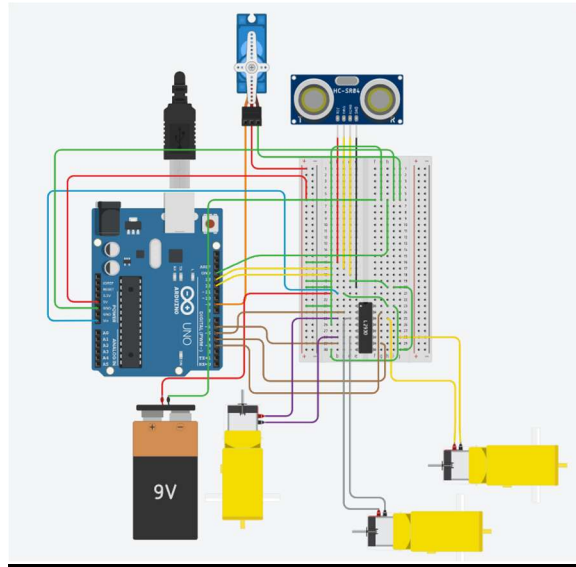




TinkerCAD Diagram



TinkerCAD Circuit



Following the template project, we modified the circuit to support 3 motors design and the servo launcher.

The circuit utilizes an L293D Motor Driver IC to control three DC hobby gearmotors.

- **Front Motors:** Motors 1 and 2 are wired in parallel to the first channel of the motor driver. This implies they will always rotate together.
- **Auxiliary Motor:** Motor 3 is wired independently to the third channel of the motor driver.

A single SG90 Micro Servo is integrated into the system

- **Control:** The signal line is connected to Arduino digital pin **D9**, which is optimal for Arduino Servo library
- **Power:** The servo draws its power directly from the Arduino's 5V rail

An HC-SR04 Ultrasonic Distance Sensor is used

- **Operation:** The sensor requires two digital pins. The Trigger pin sends the ultrasonic pulse, and the Echo pin receives the bouncing signal
- **Integration:** It is appropriately powered by the 5V rail, with the Trigger and Echo pins routed to standard digital I/O pins on the Arduino.

Arduino Source Code

Arduino Code

```

#include <Servo.h>

const int trigPin = 13;
const int echoPin = 12;
const int servoPin = 9;

// FRONT MOTORS (2 wheels in front, wired together)
// Wired to L293D Pins 2 & 7 -> Arduino Pins 5 & 6
const int frontMotorsPin1 = 5;
const int frontMotorsPin2 = 6;

// BACK MOTOR (1 single wheel in back)
// Wired to L293D Pins 10 & 15 -> Arduino Pins 4 & 3
const int backMotorPin1 = 4;
const int backMotorPin2 = 3;

// --- CONFIGURATION ---
const int WALL_DISTANCE_CM = 11;
const int REVERSE_TIME_MS = 10000;

Servo ballLauncher;

bool courseCompleted = false;

```

Snippet 1: Variables setup

- Initialize abstractions for sensor pins (trigPin and echoPin), servoPin, motors pins
- Initialize abstractions for wall distance and reverse time for easier configurations
- We need to use a reverse time as we do not have a sensor backwards so the robot does not know when to stop when moving back after finishing the ride
- Declare a Servo instance ballLauncher
- Boolean courseCompleted for later loop implementation

```

void setup() {
  Serial.begin(9600);
  // Setup Motor Pins
  pinMode(frontMotorsPin1, OUTPUT);
  pinMode(frontMotorsPin2, OUTPUT);
  pinMode(backMotorPin1, OUTPUT);
  pinMode(backMotorPin2, OUTPUT);
  // Setup Sensor & Servo
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  ballLauncher.attach(servoPin);
  // Set arm to START position (0 degrees)
  ballLauncher.write(0);
  // Wait 0.5 second before moving
  delay(500);
}

```

Snippet 2: setup() Method

- Setup every components: motors, sensor, servo
- Add a delay of 0.5 seconds before moving for plugging in the batteries

```

void loop() {
  if (courseCompleted) {
    stopMotors();
    return;
  }
  long distance = getDistance();
  Serial.print("Distance: ");
  Serial.println(distance);
  // --- OBSTACLE LOGIC ---
  if (distance > 0 && distance <= WALL_DISTANCE_CM)
  // 1. STOP at the wall
    stopMotors();
    delay(1000);
  // 2. LAUNCH BALL (100 degrees)
    ballLauncher.write(100); // Throw!
    delay(1000);
    ballLauncher.write(0); // Reset
    delay(500);
  // 3. RETURN TRIP (Go backwards)
    moveBackward();
    delay(REVERSE_TIME_MS);
  // 4. FINISH
    stopMotors();
    courseCompleted = true;
  } else {
    // Drive STRAIGHT
    moveForward();
  }

  delay(50);
}

```

Snippet 3: loop() method

- courseCompleted to check when to stop the motors
- Logic handling: if distance is less than or equal to set wall distance, the stopMotors() function will stop the motors, then wait 1 second before the servo does its work.
- The ball launcher will then be written from 0 degrees to 100 degrees to throw the ball
- After throwing, moveBackward() function would reverse the robot directions to come back to the start box
- After a set duration, the motors would stop, and mark its trip as completed
- If distance is greater than the wall distance, meaning that the robot has not reached the end, the moveForward() function will tell the robot to move straight forward.

```

void moveForward() {
    // Front 2 wheels pull forward
    digitalWrite(frontMotorsPin1, HIGH);
    digitalWrite(frontMotorsPin2, LOW);

    // Back 1 wheel pushes forward
    digitalWrite(backMotorPin1, LOW);
    digitalWrite(backMotorPin2, HIGH);
}

void moveBackward() {
    // Front 2 wheels push backward
    digitalWrite(frontMotorsPin1, LOW);
    digitalWrite(frontMotorsPin2, HIGH);

    // Back 1 wheel pulls backward
    digitalWrite(backMotorPin1, HIGH);
    digitalWrite(backMotorPin2, LOW);
}

void stopMotors() {
    digitalWrite(frontMotorsPin1, LOW);
    digitalWrite(frontMotorsPin2, LOW);
    digitalWrite(backMotorPin1, LOW);
    digitalWrite(backMotorPin2, LOW);
}

```

Snippet 4: Movement functions

- moveForward() and moveBackward(): just as their names, these functions set frontMotorsPin and backMotorsPin to its respective states to support the robot's direction movements.
- stopMotors() completely set all motors' pins to LOW state, which means stopping them all.
-

```
#include <Servo.h>
```

```
// --- PIN DEFINITIONS ---
```

```
const int trigPin = 13;
```

```
const int echoPin = 12;
```

```

const int servoPin = 9;

// --- MOTOR DEFINITIONS ---

// FRONT MOTORS (2 wheels in front, wired together)
// Wired to L293D Pins 2 & 7 -> Arduino Pins 5 & 6
const int frontMotorsPin1 = 5;
const int frontMotorsPin2 = 6;

// BACK MOTOR (1 single wheel in back)
// Wired to L293D Pins 10 & 15 -> Arduino Pins 4 & 3
const int backMotorPin1 = 4;
const int backMotorPin2 = 3;

// --- CONFIGURATION ---
const int WALL_DISTANCE_CM = 11;
const int REVERSE_TIME_MS = 10000;

Servo ballLauncher;
bool courseCompleted = false;

void setup() {
  Serial.begin(9600);

  // Setup Motor Pins
  pinMode(frontMotorsPin1, OUTPUT);
  pinMode(frontMotorsPin2, OUTPUT);
  pinMode(backMotorPin1, OUTPUT);
  pinMode(backMotorPin2, OUTPUT);

  // Setup Sensor & Servo
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  ballLauncher.attach(servoPin);

  // Set arm to START position (0 degrees)
  ballLauncher.write(0);

  // Wait 0.5 second before moving
  delay(500);
}

void loop() {
  if (courseCompleted) {
    stopMotors();
    return;
  }

  long distance = getDistance();

```

```

Serial.print("Distance: ");
Serial.println(distance);

// --- OBSTACLE LOGIC ---
if (distance > 0 && distance <= WALL_DISTANCE_CM) {

    // 1. STOP at the wall
    stopMotors();
    delay(1000);

    // 2. LAUNCH BALL (100 degrees)
    ballLauncher.write(100); // Throw!
    delay(1000);
    ballLauncher.write(0); // Reset
    delay(500);

    // 3. RETURN TRIP (Go backwards)
    moveBackward();
    delay(REVERSE_TIME_MS);

    // 4. FINISH
    stopMotors();
    courseCompleted = true;

} else {
    // Drive STRAIGHT
    moveForward();
}

delay(50);
}

// --- MOVEMENT FUNCTIONS ---

void moveForward() {
    // Front 2 wheels pull forward
    digitalWrite(frontMotorsPin1, HIGH);
    digitalWrite(frontMotorsPin2, LOW);

    // Back 1 wheel pushes forward
    digitalWrite(backMotorPin1, LOW);
    digitalWrite(backMotorPin2, HIGH);
}

void moveBackward() {
    // Front 2 wheels push backward
    digitalWrite(frontMotorsPin1, LOW);
    digitalWrite(frontMotorsPin2, HIGH);
}

```

```

// Back 1 wheel pulls backward
digitalWrite(backMotorPin1, HIGH);
digitalWrite(backMotorPin2, LOW);
}

void stopMotors() {
  digitalWrite(frontMotorsPin1, LOW);
  digitalWrite(frontMotorsPin2, LOW);
  digitalWrite(backMotorPin1, LOW);
  digitalWrite(backMotorPin2, LOW);
}

long getDistance() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Added a 30-millisecond timeout so the robot doesn't freeze!
  long duration = pulseIn(echoPin, HIGH, 30000);

  // If it times out, it returns 0. Let's treat 0 as "path is clear" (far away)
  if (duration == 0) {
    return 999;
  }

  return duration * 0.034 / 2;
}

```